EE3621: Digital Signal Processing | III B.Tech EEE

Faculty Reference Document — Textbook Replacement

PREFACE FOR FACULTY

This capstone lecture synthesizes all four units of EE3621 Digital Signal Processing. It bridges theoretical principles with real-world engineering systems by examining **Channel Equalization**, **Adaptive Noise Cancellation (ANC)**, and the **Least Mean Squares (LMS)** adaptive algorithm, followed by a comprehensive Course Outcome (CO1–CO5) mapping and revision.

Pedagogical Strategy:

1. Formulate Channel Equalization as an inverse filtering problem to eliminate Inter-Symbol Interference (ISI).
2. Formulate Adaptive Noise Cancellation (ANC) with primary input $d[n] = s[n] + n_0[n]$ and reference noise $x[n]$.
3. Derive the **Widrow-Hoff Least Mean Squares (LMS)** stochastic gradient weight update:
$$\mathbf{w}[n+1] = \mathbf{w}[n] + 2\mu e[n]\mathbf{x}[n]$$
4. Establish the LMS stability step-size condition: $0 < \mu < \frac{1}{\lambda_{\max}} < \frac{1}{\mathrm{Tr}(\mathbf{R})}$.
5. Synthesize the 4 Units of the curriculum: Signals/LTI/DTFT/Z-Transform (Unit I) $\rightarrow$ DFT/FFT/Block Filtering (Unit II) $\rightarrow$ Filter Realization Structures (Unit III) $\rightarrow$ FIR & IIR Filter Design (Unit IV).

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Formulate** channel equalization systems for dispersive digital communication links.
2. **Derive** the LMS adaptive filter algorithm from the Mean Square Error (MSE) cost function.
3. **Analyze** convergence speed vs. steady-state misadjustment trade-offs governed by step-size μ.
4. **Map** complete end-to-end DSP problem statements to the appropriate mathematical tools and algorithms developed across the course.

2. MATHEMATICAL FOUNDATIONS

2.1 Channel Equalization

When a digital symbol sequence $s[n]$ passes through a dispersive channel $C(z)$ with additive noise $v[n]$:
$$r[n] = s[n] * c[n] + v[n]$$

- **Zero-Forcing Equalizer (ZF):**

$$E_{\text{ZF}}(z) = \frac{1}{C(z)}$$

Drawback: Inverts channel nulls, causing catastrophic noise enhancement.

- **Minimum Mean Square Error (MMSE) Equalizer:**

$$E_{\text{MMSE}}(e^{j\omega}) = \frac{C^*(e^{j\omega})}{|C(e^{j\omega})|^2 + \frac{S_{vv}(\omega)}{S_{ss}(\omega)}}$$

2.2 Adaptive Noise Cancellation & The LMS Algorithm

- **System Model:**
  - Primary input: $d[n] = s[n] + n_0[n]$ ($s[n]$ is desired signal; $n_0[n]$ is noise).
  - Reference input: $x[n] = n_1[n]$ (correlated with $n_0[n]$, uncorrelated with $s[n]$).
  - Adaptive filter output: $y[n] = \mathbf{w}^T[n]\mathbf{x}[n] = \sum_{k=0}^{M-1} w_k[n]x[n-k]$.
  - Error signal: $e[n] = d[n] - y[n] = s[n] + (n_0[n] - y[n])$.
- **Cost Function:**

$$J(\mathbf{w}) = E[e^2[n]] = E[s^2[n]] + E[(n_0[n] - y[n])^2]$$

Minimizing $E[e^2[n]]$ forces $y[n] \to n_0[n]$, so $e[n] \to s[n]$.

- **LMS Weight Update Equation:**

$$\mathbf{w}[n+1] = \mathbf{w}[n] + 2\mu e[n]\mathbf{x}[n]$$

- **Stability Condition:**

$$0 < \mu < \frac{1}{\text{Tr}(\mathbf{R})} = \frac{1}{M \cdot E[x^2[n]]} = \frac{1}{MP_x}$$

2.3 Comprehensive Course Outcome (CO) Mapping

Unit	Topic Coverage	Course Outcome
Unit I (L1–L7)	DT Signals, LTI Systems, Convolution, DTFT, Z-Transform, Inverse ZT, DFT Matrix	**CO1, CO2:** Analyze discrete-time signals and transform-domain representations.
Unit II (L8–L14)	DFT Properties, Circular Conv, Radix-2 DIT/DIF FFT, Radix-4, Overlap-Add & Save	**CO2, CO3:** Implement efficient fast Fourier transforms and block linear filtering.
Unit III (L15–L20)	FIR Direct/Cascade, Linear Phase, IIR DF-I/DF-II, Cascade SOS, Parallel, Lattice	**CO4:** Synthesize robust digital filter structures and analyze quantization effects.
Unit IV (L21–L30)	FIR Windows, Frequency Sampling, Analog Prototypes, BLT, MZT, Equalization, LMS	**CO5:** Design FIR and IIR digital filters and apply them to engineering systems.

---

3. WORKED NUMERICAL EXAMPLES

Example 30.1: LMS Weight Convergence Calculation

Problem: A 2-tap adaptive LMS filter is used for noise cancellation with step size $\mu = 0.01$. The input reference noise has power $P_x = E[x^2[n]] = 2.0\text{ W}$. (a) Verify the stability of the adaptation step size. (b) Given current weights $\mathbf{w}[n] = [0.5, -0.2]^T$, reference vector $\mathbf{x}[n] = [1.0, 0.5]^T$, and primary signal $d[n] = 1.2$, compute the updated weights $\mathbf{w}[n+1]$.

Solution: (a) Stability Check: Maximum stable step size:

$$\mu_{\max} = \frac{1}{MP_x} = \frac{1}{2 \times 2.0} = \frac{1}{4.0} = 0.25$$

Since $\mu = 0.01 < 0.25$, the adaptive algorithm is **guaranteed to be stable and converge**.

(b) Weight Update:

1. Filter Output:

$$y[n] = \mathbf{w}^T[n]\mathbf{x}[n] = 0.5(1.0) + (-0.2)(0.5) = 0.5 - 0.1 = 0.4$$

2. Error Signal:

$$e[n] = d[n] - y[n] = 1.2 - 0.4 = 0.8$$

3. Weight Update:

$$\mathbf{w}[n+1] = \mathbf{w}[n] + 2\mu e[n]\mathbf{x}[n] = [0.5 \;\; -0.2] + 2(0.01)(0.8)\,[1.0 \; 0.5] = [0.5 \;\; -0.2] + 0.016\,[1.0 \; 0.5]$$
$$\mathbf{w}[n+1] = [0.5 + 0.0160 \;\; -0.2 + 0.0080] = [\mathbf{0.5160} \;\; \mathbf{-0.1920}]$$

---

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) Explain the principle of Adaptive Noise Cancellation (ANC). Derive the LMS weight adaptation algorithm and state the condition for convergence. *(9 Marks)* **(b)** What is channel equalization? Compare Zero-Forcing and MMSE equalizers. *(6 Marks)*

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
  - ANC block diagram and proof that minimizing $E[e^2[n]]$ maximizes signal SNR *(4 Marks)*
  - Derivation of LMS gradient update $\mathbf{w}_{n+1} = \mathbf{w}_n + 2\mu e_n \mathbf{x}_n$ *(3 Marks)*
  - Stability bounds: $0 < \mu < 1/\mathrm{Tr}(\mathbf{R})$ *(2 Marks)*
- **Part (b):**
  - Channel distortion and ISI formulation *(2 Marks)*
  - Comparison of ZF ($1/C(z)$, noise boost) vs MMSE (noise regularization) *(4 Marks)*

---

5. PYTHON VERIFICATION SCRIPT

```
import numpy as np

# LMS simulation
np.random.seed(42)
N_samples = 500
s = np.sin(2 * np.pi * 0.05 * np.arange(N_samples))  # Desired signal
n0 = 0.5 * np.random.randn(N_samples)               # Noise
d = s + n0                                          # Primary input
x = n0 + 0.1 * np.random.randn(N_samples)           # Reference noise

# LMS Adaptive Filter
M = 4
mu = 0.01
w = np.zeros(M)
e = np.zeros(N_samples)

for n in range(M, N_samples):
    x_vec = x[n:n-M:-1]
    y = np.dot(w, x_vec)
    e[n] = d[n] - y
    w = w + 2 * mu * e[n] * x_vec

print("Final LMS Weights:", np.round(w, 4))
print(f"Output Error variance reduction: {np.var(d):.4f} → {np.var(e[100:]):.4f}")
```